



Rúbrica de Evaluación - Semana 06

Custom Hooks y Proyecto Integrador



Información General

Aspecto	Detalle
Semana	06 de 20
Etapas	2 - Fundamentos de React (Final)
Tema Principal	Custom Hooks y Proyecto Integrador
Duración	8 horas
Tipo Evaluación	Sumativa (proyecto 100%)



Objetivos Evaluables

1. Crear custom hooks reutilizables con TypeScript
2. Extraer lógica de componentes a hooks personalizados
3. Tipar hooks con generics para máxima flexibilidad
4. Integrar todos los conceptos de la Etapa 2 en un proyecto

Recursos de Aprendizaje (No Evaluados)

La teoría y los ejercicios son recursos de preparación para el proyecto. No son entregables evaluados, pero se recomienda completarlos para adquirir las competencias necesarias.

Teoría

- Custom hooks: fundamentos y anatomía
- Patrones de composición de hooks
- Ciclo de vida de componentes
- Configuración de Vite

Ejercicios Guiados

- Ejercicio 01: useToggle , useCounter - Hooks básicos
- Ejercicio 02: useForm , useArray - Estado complejo
- Ejercicio 03: useFetch , useDebounce - Efectos
- Ejercicio 04: Hooks genéricos con <T>
- Ejercicio 05: Composición de múltiples hooks



Proyecto (100%)

Instrumento: Dashboard Interactivo (Proyecto Etapa 2)

Funcionalidad (40%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
Custom hooks implementados	5+ hooks reutilizables	3-4 hooks	2 hooks	<2 hooks

Widgets interactivos	4+ widgets funcionales	3 widgets	2 widgets	<2 widgets
Context API (tema/usuario)	2+ contexts bien tipados	1 context completo	Context básico	Sin context
Manejo de estados	Loading/error/data	Estados básicos	Falta algún estado	No maneja estados

Adaptación al Dominio (35%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
Entidades del dominio	Todas coherentes	Mayoría coherente	Parcialmente coherente	Incoherente
Widgets adaptados	Específicos del dominio	Adaptados	Genéricos	Sin adaptar
Nomenclatura contextualizada	Nombres del dominio	Parcialmente	Pocos nombres	Genérica
Datos mock representativos	Realistas para el dominio	Aceptables	Básicos	Sin relación

Calidad del Código (25%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
TypeScript estricto	0 errores, tipos completos	Errores menores	any ocasional	Muchos any
Estructura de código	Separación clara, reutilizable	Organizado	Algo desorganizado	Desordenado
Documentación	JSDoc completo	Comentarios	Mínima	Sin documentar
Hooks bien encapsulados	Single responsibility	Aceptable	Hacen demasiado	No encapsulados

1234

Cálculo de Calificación

Calificación Final = (Funcionalidad × 0.40) + (Adaptación al Dominio × 0.35) + (Calidad del Código × 0.25)

Escala de Conversión

Puntos	Calificación	Nivel
95-100	10	Excepcional
85-94	9	Sobresaliente
75-84	8	Notable
70-74	7	Aprobado
60-69	6	Suficiente
< 60	5 o menos	No aprobado

Criterios de Aprobación

- Mínimo 70% en el proyecto para aprobar la semana
- Proyecto funcional y ejecutable
- Adaptación coherente al dominio asignado

Checklist de Entrega

Preparación (No evaluada)

- ☐ Ejercicio 01: `useToggle` , `useCounter` implementados
- ☐ Ejercicio 02: `useForm` , `useArray` con estado complejo
- ☐ Ejercicio 03: `useFetch` , `useDebounce` con efectos
- ☐ Ejercicio 04: Hooks genéricos con `<T>`
- ☐ Ejercicio 05: Composición de múltiples hooks

Proyecto Dashboard (100% - Evaluado)

- ☐ Repositorio con estructura organizada
- ☐ README con descripción del dominio asignado
- ☐ Mínimo 5 custom hooks reutilizables:
 - ☐ Hook de estado (toggle, counter, form)
 - ☐ Hook de efectos (fetch, debounce)
 - ☐ Hook de storage (localStorage)
 - ☐ Hook de UI (responsive, theme)
 - ☐ Hook compuesto (auth, dashboard data)
- ☐ Mínimo 3 widgets funcionales
- ☐ Context para tema (claro/oscurο)
- ☐ TypeScript sin errores (`tsc --noEmit`)
- ☐ Código documentado con comentarios JSDoc

Adaptación por Dominio

El proyecto debe estar adaptado al dominio asignado:

Dominio	Widgets Sugeridos
 Biblioteca	Libros prestados, usuarios activos, búsqueda
 Farmacia	Stock bajo, ventas del día, alertas vencimiento
 Gimnasio	Miembros activos, clases hoy, ocupación
 Escuela	Estudiantes, promedio notas, asistencia
 Pet Shop	Productos populares, citas grooming, inventario
 Restaurante	Pedidos activos, mesas ocupadas, platos populares
 Banco	Transacciones hoy, saldo total, alertas
 Taxis	Viajes activos, conductores disponibles, ingresos
 Hospital	Pacientes hoy, camas disponibles, emergencias

17 Fechas Importantes

Actividad	Fecha Límite
Preparación (ejercicios)	Recomendado día 5
Proyecto Dashboard	Día 7 de la semana
Retroalimentación	Día 8-9

Nota: Solo el proyecto es entregable evaluado. Los ejercicios son preparación recomendada.

Retroalimentación

Fortalezas Comunes

- Hooks bien encapsulados y reutilizables
- Tipado correcto con generics
- Buena separación de concerns

Áreas de Mejora Frecuentes

- Olvidar cleanup en useEffect dentro de hooks
- No manejar todos los estados (loading, error)
- Hooks que hacen demasiadas cosas
- Dependencias incorrectas en useEffect

Recursos de Refuerzo

- [React Docs: Custom Hooks](#)
- [useHooks.com](#) - Colección de hooks
- [ahooks](#) - Librería de hooks

Competencias Etapa 2 Completadas

Al aprobar esta semana, el estudiante demuestra:

Competencia	Nivel Alcanzado
Componentes funcionales TypeScript	✅ Intermedio
Hooks nativos (useState, useEffect)	✅ Intermedio
Manejo de eventos y formularios	✅ Intermedio
Renderizado condicional y listas	✅ Intermedio
Context API	✅ Básico
Custom Hooks	✅ Básico
Configuración Vite	✅ Básico

Próximo paso: Etapa 3 - React Intermedio (React Router, Zustand, React Query)